

CS306 — Database Systems

Project Phase 4

Smart Parking Reservation Database System

Web Integration of SQL and NoSQL Databases

Group Number:	01
Group Members:	İsmail Memiş
Database:	parking_db2 (MariaDB 10.4)
Stack:	PHP 8 · MySQLi · MongoDB · XAMPP
Submission:	Phase 4

1. Introduction

This report documents Phase 4 of the Smart Parking Reservation Database System project. The objective of this phase is to extend the relational database built in earlier phases into a working web application that integrates a MySQL trigger and stored procedure through PHP, and that adds a NoSQL support-ticket subsystem on MongoDB. The application is hosted locally through XAMPP and is split into two interfaces: a user interface at localhost/user and an admin interface at localhost/admin.

The trigger and stored procedure used in this phase were designed during Phase 2 and are now exposed to the user through interactive web pages. The support-ticket system is backed entirely by MongoDB and supports ticket creation, listing, commenting and resolution.

2. Trigger Integration

2.1 Description

The trigger `trg_prevent_overlap_reservation` fires **BEFORE INSERT** on the `RESERVATION` table. Its purpose is to enforce a business rule that cannot be expressed with a simple foreign-key or check constraint: no two non-cancelled reservations may overlap in time on the same parking spot. If the requested time range conflicts with an existing active or completed reservation on the same spot, the trigger raises SQLSTATE '45000' and the insert is rejected. Otherwise, when the new reservation is active, the trigger also updates the corresponding `PARKING_SPOT.status` to 'occupied'.

2.2 SQL Definition

```
DELIMITER $$
```

```
CREATE TRIGGER trg_prevent_overlap_reservation
BEFORE INSERT ON RESERVATION
FOR EACH ROW
BEGIN
```

```
    DECLARE overlap_count INT DEFAULT 0;
```

```
    IF NEW.status <> 'cancelled' THEN
```

```
        SELECT COUNT(*) INTO overlap_count
        FROM RESERVATION R
        WHERE R.spot_id = NEW.spot_id
            AND R.status IN ('active', 'completed')
            AND NEW.start_time < R.end_time
            AND NEW.end_time > R.start_time;
```

```
    IF overlap_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Reservation conflict: this spot is already booked.';
    END IF;
```

```
    IF NEW.status = 'active' THEN
        UPDATE PARKING_SPOT
        SET status = 'occupied'
        WHERE spot_id = NEW.spot_id;
    END IF;
```

```

        END IF;
    END$$

    DELIMITER ;

```

2.3 Cases on the Web Page

The trigger page (user/trigger_page.php) exposes three buttons. Each one drives the trigger into a different execution path so that all branches of its logic can be observed.

Button	Scenario	Expected outcome
Case 1	Insert reservation #9001 on spot 7 (currently available), successful trigger sets spot 7 status to 'occupied'.	able, successful trigger sets spot 7 status to 'occupied'.
Case 2	Insert reservation #9002 on spot 3, time window (2026-03-04 11:00:00-13:00:00) overlaps with existing reservation (2026-03-04 11:00:00-13:00:00) on spot 3, fails with SQLSTATE 45000 reservation conflict.	2026-03-04 11:00:00-13:00:00) overlaps with existing reservation (2026-03-04 11:00:00-13:00:00) on spot 3, fails with SQLSTATE 45000 reservation conflict.
Case 3	Reset: removes test reservations and sets spot 7 back to available so the previous two cases can be re-run.	Back to available so the previous two cases can be re-run.

Screenshot placeholder: Insert a screenshot of the trigger page after pressing Case 1 (success message + spot status = 'occupied') and Case 2 (red error message from SQLSTATE 45000).

3. Stored Procedure Integration

3.1 Description

The stored procedure `sp_GetDriverReservationStats(p_driver_id)` produces an analytical summary for a single driver. It first verifies that the driver exists (raising `SQLSTATE 45000` otherwise) and then returns one row containing: the driver's name, the total number of reservations, a status breakdown (active / completed / cancelled), and the parking lot the driver uses most frequently. The most-used lot is computed by an inner subquery that joins `RESERVATION` with `PARKING_SPOT` and `PARKING_LOT`, groups by lot, and orders by count.

3.2 SQL Definition

```
DELIMITER $$

CREATE PROCEDURE sp_GetDriverReservationStats(IN p_driver_id INT)
BEGIN
    DECLARE v_exists INT DEFAULT 0;

    SELECT COUNT(*) INTO v_exists
    FROM DRIVER WHERE driver_id = p_driver_id;

    IF v_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Driver not found with the given driver_id.';
    END IF;

    SELECT
        D.driver_id,
        D.name AS driver_name,
        COUNT(R.reservation_id) AS total_reservations,
        SUM(CASE WHEN R.status='active' THEN 1 ELSE 0 END) AS active_count,
        SUM(CASE WHEN R.status='completed' THEN 1 ELSE 0 END) AS completed_count,
        SUM(CASE WHEN R.status='cancelled' THEN 1 ELSE 0 END) AS cancelled_count,
        (
            SELECT PL.lot_name
            FROM RESERVATION R2
            JOIN PARKING_SPOT PS ON R2.spot_id = PS.spot_id
            JOIN PARKING_LOT PL ON PS.lot_id = PL.lot_id
            WHERE R2.driver_id = D.driver_id
            GROUP BY PL.lot_id, PL.lot_name
            ORDER BY COUNT(*) DESC
            LIMIT 1
        ) AS most_used_lot
    FROM DRIVER D
    LEFT JOIN RESERVATION R ON D.driver_id = R.driver_id
    WHERE D.driver_id = p_driver_id
    GROUP BY D.driver_id, D.name;
END$$

DELIMITER ;
```

3.3 Web Page and Input

The procedure page (`user/procedure_page.php`) provides a single input field and a submit button:

Input box	Parameter	Type	Example value
-----------	-----------	------	---------------

Driver ID	p_driver_id	INT	1 – 6
-----------	-------------	-----	-------

After clicking **Call Procedure**, the PHP page binds the submitted ID into a prepared statement, executes `CALL sp_GetDriverReservationStats(?)`, and renders the returned row as an HTML table.

Screenshot placeholder: Insert a screenshot of the procedure page after submitting driver_id = 1, showing the summary table (total = 3, active = 1, completed = 2, cancelled = 0, etc.).

4. Support Ticket System (MongoDB)

All support-ticket data is persisted in a single MongoDB collection, `tickets.entries`. Each document has the following fields:

Field	Type	Notes
<code>_id</code>	ObjectId	Auto-generated MongoDB identifier.
<code>username</code>	string	Creator of the ticket.
<code>body</code>	string	Ticket message.
<code>created_at</code>	string	Creation timestamp (Y-m-d H:i:s).
<code>status</code>	boolean	true = active, false = resolved.
<code>comments</code>	array	Array of {username, comment, created_at}.

4.1 MongoDB Queries — PHP Snippets

(a) Listing usernames that have at least one active ticket (populates the dropdown on `tickets.php`).

```
$command = new MongoDB\Driver\Command([
    'distinct' => 'entries',
    'key'      => 'username',
    'query'    => ['status' => true],
]);
$cursor = $manager->executeCommand('tickets', $command);
$res     = current($cursor->toArray());
$active_usernames = $res->values ?? [];
```

(b) Fetching the active tickets of a selected user.

```
$query = new MongoDB\Driver\Query(
    ['username' => $selected_user, 'status' => true],
    ['sort'     => ['created_at' => -1]]
);
$cursor = $manager->executeQuery('tickets.entries', $query);
```

(c) Creating a new ticket from `create_ticket.php`.

```
$bulk = new MongoDB\Driver\BulkWrite;
$bulk->insert([
    'username' => $username,
    'body'     => $body,
    'status'   => true,
    'created_at' => date('Y-m-d H:i:s'),
    'comments' => [],
]);
$manager->executeBulkWrite('tickets.entries', $bulk);
```

(d) Adding a comment to a ticket (used on both user and admin detail pages).

```
$bulk = new MongoDB\Driver\BulkWrite;
$bulk->update(
    ['_id' => new MongoDB\BSON\ObjectId($id)],
    ['$push' => [
        'comments' => [
            'username' => $c_user,
```

```

        'comment'    => $c_text,
        'created_at' => date('Y-m-d H:i:s'),
    ]
    ]
);
$manager->executeBulkWrite('tickets.entries', $bulk);

```

(e) Admin-only: marking a ticket as resolved.

```

$bulk = new MongoDB\Driver\BulkWrite;
$bulk->update(
    ['_id' => new MongoDB\BSON\ObjectId($id)],
    ['$set' => ['status' => false]]
);
$manager->executeBulkWrite('tickets.entries', $bulk);

```

(f) Admin homepage: listing every active ticket across all users.

```

$query = new MongoDB\Driver\Query(
    ['status' => true],
    ['sort'    => ['created_at' => -1]]
);
$cursor = $manager->executeQuery('tickets.entries', $query);

```

Screenshot placeholder: Insert MongoDB Compass screenshots of the tickets.entries collection showing both active and resolved documents, plus screenshots of the user ticket list, the create-ticket page, the user detail page, and the admin detail page (with Deactivate Ticket button).

5. Support Ticket Workflow

The end-to-end workflow of the support system is:

1. A user opens `tickets.php`, clicks *Create a Ticket*, fills in their username and message, and submits the form. A new document is inserted into `tickets.entries` with `status: true` and an empty `comments` array.
2. The user returns to `tickets.php` and selects their username from the dropdown. Their active tickets appear, each linking to its detail page.
3. On the detail page, the user (or anyone with the link) may add comments. New comments are pushed into the `comments` array using `$push`.
4. An admin opens `localhost/admin`, which lists every active ticket regardless of user, and clicks on one to view its details. The admin can add their own comments (automatically tagged as `username: 'admin'`) and may press **Deactivate Ticket** to set `status: false`.
5. Once resolved, the ticket no longer appears on either the user or admin homepage, because both pages filter by `status: true`.

6. Assumptions and Design Decisions

- **Authentication.** The project specification treats the username dropdown as a lightweight login system; no password layer is implemented and the username field on the create-ticket page is taken on trust.
- **Foreign keys vs. trigger.** The overlap rule is enforced in the trigger because no foreign-key combination can express a temporal overlap. Cancelled reservations are explicitly excluded from the check, since a cancelled booking no longer reserves the spot.
- **Prepared statements.** All MySQL calls that take user input (the stored-procedure parameter) use `mysqli` prepared statements with `bind_param` to prevent SQL injection.
- **MongoDB driver.** Following the specification, all MongoDB access goes through `MongoDB\Driver\Manager`, `Query`, `Command`, and `BulkWrite` rather than the high-level `MongoDB\Client` helper.
- **Idempotent trigger test.** The trigger page provides a *Reset* case so that the success and failure scenarios can be replayed during the live demo without manually editing the database.

7. Project Structure

```
Group01_phase4/
├── Report.pdf
├── SQLDump.sql
├── Scripts/
│   ├── user/
│   │   ├── db.php
│   │   ├── mongo.php
│   │   ├── index.php
│   │   ├── trigger_page.php
│   │   ├── procedure_page.php
│   │   ├── tickets.php
│   │   ├── create_ticket.php
│   │   └── ticket_detail.php
│   └── admin/
```



```
|— mongo.php
|— index.php
|— ticket_detail.php
```

After extracting the archive, copy Scripts/user into xampp/htdocs/user and Scripts/admin into xampp/htdocs/admin. Run composer require mongodb/mongodb in each of those folders so that vendor/autoload.php exists. Import SQLDump.sql into phpMyAdmin and start MongoDB. The interfaces are then available at localhost/user and localhost/admin.